

Alps Alpine Force Sensor Board (Qwiic) - Application Notes

These are the application notes for the Alps Alpine Force Sensor Board (Qwiic).

I2C Address Assignments

A0	A1	I2C Address
0	0	0x74 (Default)
0	1	0x75
1	0	0x76
1	1	0x77

Message Sequence & Format

All Alps Alpine QWIIC Sensor/HMI boards use a similar I²C communication sequence between the master and slave. In these exchanges, the Alps Alpine QWIIC Sensor/HMI boards always function as the slave device.

The master is responsible for initiating both read and write operations. To change a setting or request data from a board, follow the sequence below:

**** Placeholder Sequence. Add diagram later ****

- The Master must first send request message → The board will prepare the data or change setting as requested
- The Master must then send read message with size required for the response → The board will send back the requested data response

Request Message Types

The following message types are used to indicate whether the board should prepare requested data for the user to read back or execute an operation. This message type ID is located at the beginning of the message payload sent from the I2C master device to the Alps Alpine Sensor/HMI board.

Request type	Message ID Value (uint8)	Description
Request Data	0x01	Used to request the board to prepare data that is in the data request (proceeding value in the request payload)
Request Operation	0x02	Used to request the board to execute an operation (proceeding operation in the request payload)

Request Data Types & Formats (0x01):

The following are different request data types that are supported by the Alps Alpine Force Sensor Board (Qwiic) and their expected formats.

“WHO AM I” Data Request Message (0x01):

The “WHO AM I” message can be used to both determine if an Alps Alpine Sensor/HMI board is present and can be used to determine what type of Sensor/HMI board it is.

Request I2C Payload:

Byte Index	Value	Description
0	0x01	Request Data type
1	0x01	Who Am I Data Request

Response I2C Payload:

During response request for “WHO AM I”, Master must perform a read request with **3 bytes**.

Byte Index	Value	Description
0	0x01	Message Header
1	0x01	Message Header
2	0x05	Device type indicator. 0x05 = Alps Alpine Force Sensor Board

		(Qwiic)
--	--	---------

“Get Formatted Values” Data Request Message (0x02):

The “Get formatted values” message, can be used to request the current values from the Sensor board in a more friendly format for reading. Response data and its format may change based on current set modes. Presents the user with a formatted value presented as force (newtons).

Request I2C Payload:

Byte Index	Value	Description
0	0x01	Request Data type
1	0x02	Request Formatted Values

Response I2C Payload:

In default operating mode, during the response request for “Get Formatted Values Data”, the Master must perform a read request with **4 bytes**.

Byte Index	Content	Description
0 – 3	<code>forceValueN</code> (<code>float</code> , little-endian)	Sensor's force reading Unit: Newton “N”

“Get Raw Values” Data Request Message (0x03):

The “Get Raw values” message, can be used to request the current raw data that the Sensor board reads from the Alps Alpine component after it goes through the differential amplifier. This raw data represents a voltage value that's output from the differential amplifier after the current set [Zero Calibration](#) offset is applied to it. [More details about how to use and convert this raw value into formatted force value can be found below.](#)

Request I2C Payload:

Byte Index	Value	Description
0	0x01	Request Data type
1	0x03	Request Raw Values

Response I2C Payload:

In default operating mode, during the response request for “Get Raw Values Data”, the Master must perform a read request with **4 bytes**. The value presented in this request is the raw amplified differential analog value that's read from the amplifier component connected to the sensor's output pins.

Byte Index	Content	Description
0 – 3	<code>rawForceVoltageValue</code> (<code>float</code> , little-endian)	Raw voltage reading from the differential amplifier. ** Zero-Calibration offset is already pre-applied to this value

“Get Is Data Ready” Data Request Message (0x04):

The “Get Is Data Ready” message can be used to check whether the Sensor board is ready to provide component data readings when requested. It is good practice to perform this request after powering on the board and before requesting any data, continuing until the board indicates that it is ready. This message should also be used after any operation request to wait until the board is ready before sending additional requests.

Request I2C Payload:

Byte Index	Value	Description
0	0x01	Request Data type
1	0x04	Request Is Data Ready Value

Response I2C Payload:

In default operating mode, during the response request for “Get Is Data Ready”, the Master must perform a read request with **1 byte**.

Byte Index	Content	Description
0	<code>isDataReady</code>	0x00 = Data Not Ready

	(uint8_t)	• 0x01 = Data Ready
--	-------------	----------------------------

“Get Firmware Version” Data Request Message (0x05):

The “Get Firmware Version” message can be used to check the firmware version of the Sensor board.

Request I2C Payload:

Byte Index	Value	Description
0	0x01	Request Data type
1	0x05	Request Firmware Version

Response I2C Payload:

In default operating mode, during the response request for “Get Firmware Version”, the Master must perform a read request with **2 bytes**.

Byte Index	Content	Description
0	MAJOR_VERSION (uint8_t)	The firmware's major value
1	MINOR_VERSION (uint8_t)	The firmware's minor value

“Get Current Calibration Values” Data Request Message (0x06):

The “Get Current Calibration Values” message can be used to get the currently used “Calibration Values” used by the Sensor board to perform a more accurate force value calculation.

Request I2C Payload:

Byte Index	Value	Description
0	0x01	Request Data type
1	0x06	Request Current Calibration Values

Response I2C Payload:

During the response request for “Get Current Calibration Values”, the Master must perform a read request with **13 bytes**. The received calibration values are calculated during calibrations and are automatically applied to the data requested in varying ways. These values can also be saved externally and set on power up (using the “[Set Calibration Values](#)” message) to re-use known calibration values and avoid needing to perform during each power up. [More detail on how to manually apply these values to a raw force value can be found below.](#)

Byte Index	Content	Description
0 - 3	M (float , little-endian)	M slope value (N/Volts)
4 - 7	B (float , little-endian)	B offset / intercept (N)
8 - 11	zeroRefV (float , little-endian)	The zero force offset voltage (Volts)
12	isSpanValValid (uint8_t)	• 0x00 = False • 0x01 = True

Request Operation Types & Formats (0x02):

The following are different request operation types that are supported by the Alps Alpine Force Sensor Board (Qwiic) and their expected formats. These requests do not have any responses in return.

“Perform Zero Calibration” Operation Request Message (0x01):

The “Perform Zero Calibration” message can be used to request, over the I2C interface, the Sensor Board to start a “Zero Calibration”. [A full description about how Zero Calibration works and how to perform it can be found below.](#)

Request I2C Payload:

Byte Index	Value	Description
0	0x02	Request Operation type

1	0x01	Request Perform Zero Calibration
---	------	----------------------------------

“Perform Span-Calibration” Operation Request Message (0x02):

The “Perform Span-Calibration” message can be used to request, over the I2C interface, the Sensor board to start a “Span-Calibration”. [A full description about how Span-Calibration works and how to perform it can be found below.](#)

Request I2C Payload:

Byte Index	Value	Description
0	0x02	Request Operation type
1	0x02	Request Perform Span-Calibration
2 - 5	knownForceN (float , little-endian)	Known Force Value for Calibration Process ** (It is recommended to be between 7N - 10N)

“Set Calibration Values” Operation Request Message (0x03):

The “Set Calibration Values” message can be used to request, over the I2C interface, the Sensor board to save a set of previously calculated calibration values. This allows the user to manually perform calibrations on their own and externally save values to be set during power up of the sensor board.

Request I2C Payload:

Byte Index	Value	Description
0	0x02	Request Operation type
1	0x03	Set Calibration Values

2 - 5	M (float , little-endian)	M slope value (N/Volts)
6 - 9	B (float , little-endian)	B offset / intercept (N)
10 - 13	zeroRefV (float , little-endian)	The zero force offset voltage (Volts)
14	isSpanValValid (uint8_t)	Flag to set if the Span-Calibration values should be used. If valid values are not set, mark flag as False . • 0x00 = False • 0x01 = True

“Enter DFU Mode” Operation Request Message (0xff):

The “Enter DFU Mode” message can be used to request the Sensor board to enter its DFU mode. This allows for an external I2C Master device to perform DFU over I2C interface.

Request I2C Payload:

Byte Index	Value	Description
0	0x02	Request Operation type
1	0xff	Request Enter DFU Mode

How To Perform Calibrations:

The force sensor board incorporates two calibration methods that can be used in order to create a more accurate force calculation and negate any irregularities due to electrical properties and external environmental factors. These calibrations save the current calibration values to be used when calculating force and automatically applied to the formatted force value. Additionally, the

current Zero Calibration offset value is pre-applied to the raw value that is received when making a “[Get Raw Values](#)” data request.

The sensor board does not however save these calibration values into saved flash and on power up, Zero Calibration value is set to default (0.762V) and Span-Calibration is cleared and set to disabled. After power-up, calibration must be re-done or externally saved values can be set using the “[Set Calibration Values](#)” operation message.

Zero Calibration:

Zero calibration sets the current no-load sensor output as the baseline, so the device treats that resting value as 0 N instead of interpreting built-in sensor and amplifier offset as force. Before a manual zero calibration is performed, the default offset value is set to **0.762V** (calculated by built-in HW components). Before performing, make sure nothing is touching or loading the sensor, keep the device still, then run the “[Perform Zero Calibration](#)” command so the firmware stores the current reading as the zero reference.

**** Additionally, any currently saved Span-Calibration values will be cleared and must be performed again, since the span calculation depends on the newly established zero-calibration reference.**

Span-Calibration:

Span-Calibration sets the force scale by measuring the sensor output while a known load is applied, allowing the firmware to calculate how many newtons correspond to the measured voltage change.

Force calculation with Span-Calibration applied: $\text{Force[N]} = M * (\text{Vout} - \text{REF}) + B$

M = Slope value (N/Volts)

$(\text{Vout} - \text{REF})$ = Difference between Amplifier output and zero offset reference (This whole value represents the value acquired from “[Get Raw Values](#)” data request)

B = Offset / intercept (N)

To perform it, first complete [Zero Calibration](#) with no load on the sensor, then apply the specified known force steadily and then run the “[Perform Span-Calibration](#)” command so that reading is stored as the calibration slope.

Convert Raw Force Value (V) to Formatted Force (N):

When requesting to get raw force value, this value represents the voltage output from the differential amplifier after it converts and boosts the output from the force sensor component. This value already has any set zero offset value, acquired during a Zero Calibration event or using default calculated values, applied to it.

Conversion can be done with or without a valid set of Span-Calibration values acquired by the [“Get Current Calibration Values”](#) message. The conversion can be performed using the following C++ code block:

```
1  #include <cmath>
2
3  struct ForceSpanValues {
4      float M = 0.0f; // N/Volt
5      float B = 0.0f; // N
6  };
7
8  inline float convertRawForceSensorBoardData(float rawValue, bool isSpanValuesValid, const
ForceSpanValues& spanValues) {
9
10     // If Span-Calibration is valid, use the calibrated linear mapping.
11     if (isSpanValuesValid) {
12
13         //Calculate Force [N] using the Span-Calibration values
14         float forceN = spanValues.M * rawValue + spanValues.B;
15
16         //Clip noise
17         if (forceN < 0.0f && std::fabs(forceN) < 0.2f) {
18             forceN = 0.0f;
19         }
20
21         return forceN;
22     }
23
24     // Otherwise fall back to the nominal hardware-based conversion.
25     static constexpr float ampGain = 1.0f + 100000.0f / 4020.0f;
26     static constexpr float alphaBridgeVPerN = 2.3e-3f * 3.3f;
27     static constexpr float alphaAmpVPerN = alphaBridgeVPerN * ampGain;
28
29     float denom = std::fabs(alphaAmpVPerN) < 1e-6f ? 1e-6f : alphaAmpVPerN;
30     float forceN = rawValue / denom;
31
32     //Clip noise
33     if (forceN < 0.0f && std::fabs(forceN) < 0.2f) {
34         forceN = 0.0f;
35     }
36
37     return forceN;
38 }
```